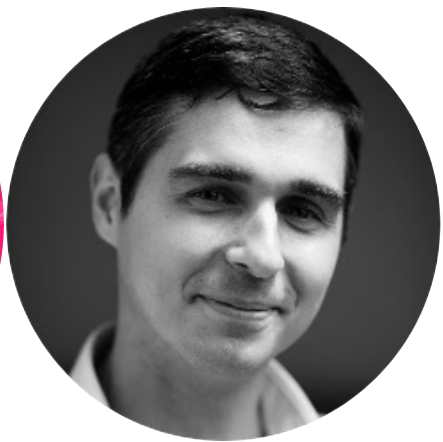


Java SE 21 Developer (Exam 1Z0-830): Exception Handling

Introducing Exception Handling



Andrejs Doronins

Software Developer in Test



```
try {  
    // read a file  
} catch (IOException ex) {  
    // handle  
}  
finally { ... }  
  
if(/* invalid input */){  
    throw new IllegalArgumentException("ouch...");  
}
```



Valid Code?

App.java

```
if(checkSomething()){  
    throw RuntimeException();  
}
```

App2.java

```
try {  
    // do stuff  
} catch (IllegalArgumentException  
ex) {  
    // handle  
} catch (NumberFormatException ex)  
{  
    // handle  
}
```



Be the compiler...



Course Introduction

Advantages of exception handling

Rules of try / catch / finally

Practicing try-with-resources

Exception types

- Exception class hierarchy

Throwing

- Custom exceptions



It's a Dangerous World

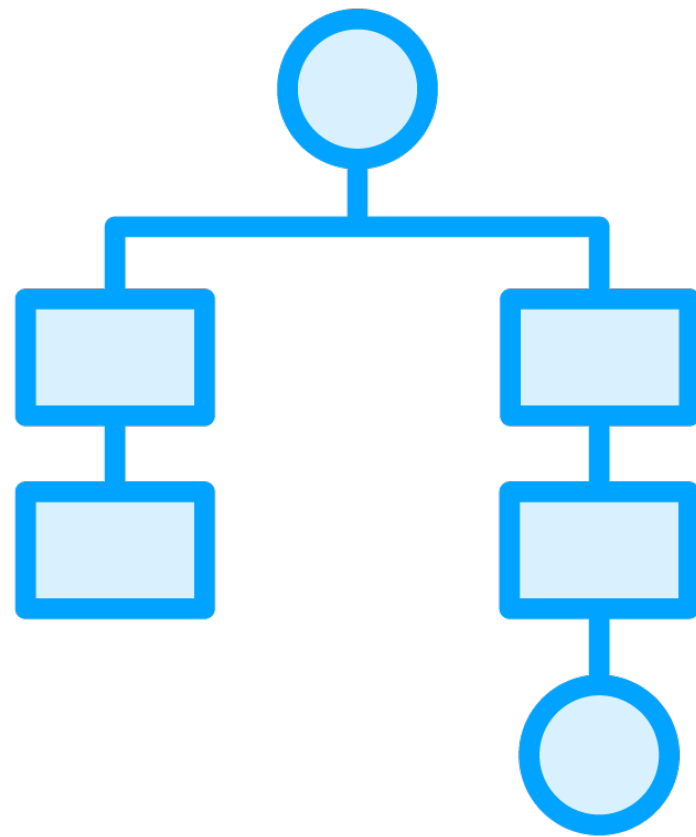
Internet is down

No disk space left

Access failure

Empty array





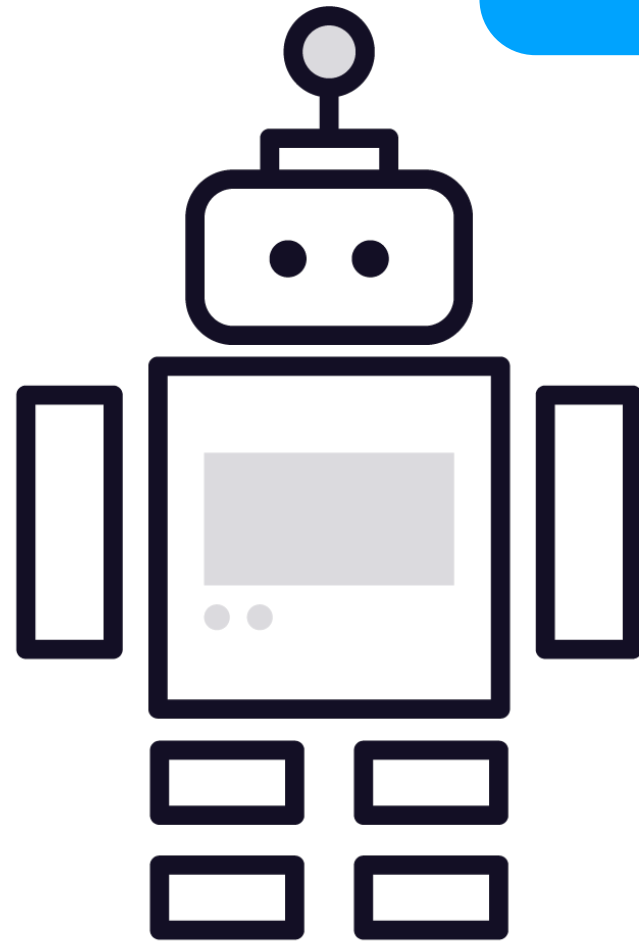
Errors:

- Within your control
- Outside of your control

Exam questions focus on exceptions caused by programming mistakes and errors (within your control)

Will this compile?

I give up. I don't know what to do right now.
You deal with it!



System.exit(-1);

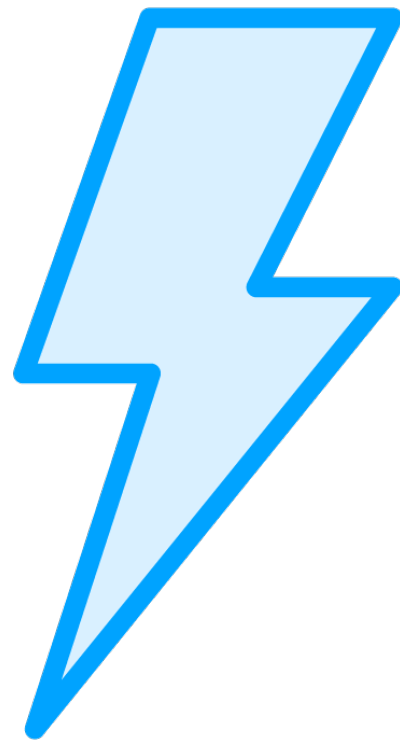
Error codes instead exceptions (e.g. -1)

But what went wrong exactly?

Error: -4?

- File can't be opened?
- Enough memory can't be allocated?





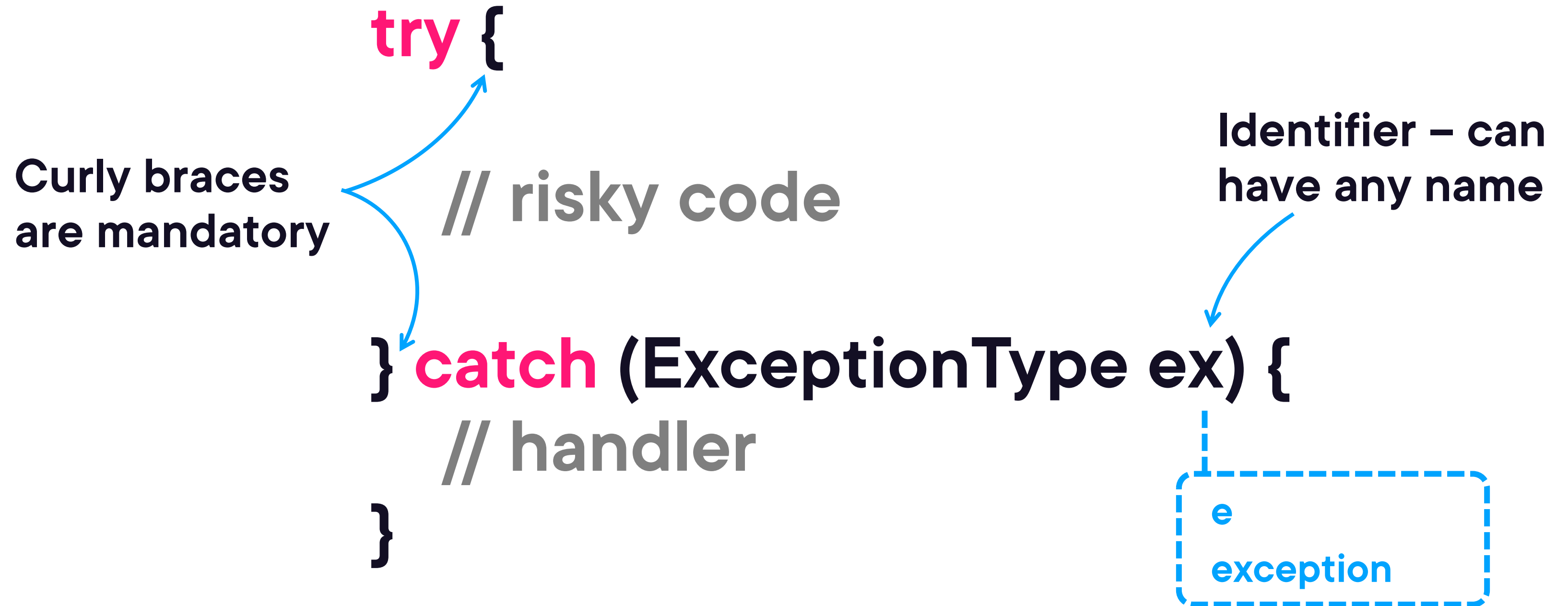
Exceptions:

- Specific names for specific errors
- Example: `NumberFormatException`
 - “50” -> 50
 - “1a” -> ??

Special syntax to deal with exceptions:

- Try/catch/finally





Interchangeable Terms

Block

Clause



```
BufferedReader br;  
try {  
    br = new BufferedReader(new FileReader("file.txt"));  
    String line;  
    while ((line = br.readLine()) != null) {  
        System.out.println(line);  
    }  
} catch (IOException) {  
    System.err.format("Exception happened");  
}
```



Will This Compile?

catch or finally is mandatory

```
try {  
    doRiskyStuff();  
} 
```



```
doTheOtherThing();
```



Will This Compile?

Braces are
mandatory



```
try  
    openFile("file.txt");  
catch (IOException e)  
    System.out.println(e);
```

Braces are optional
for if-else blocks



```
if (canReadFile("file.txt"))  
    openFile("file.txt");  
else  
    System.out.println("error!");
```



Object

...

ParentException

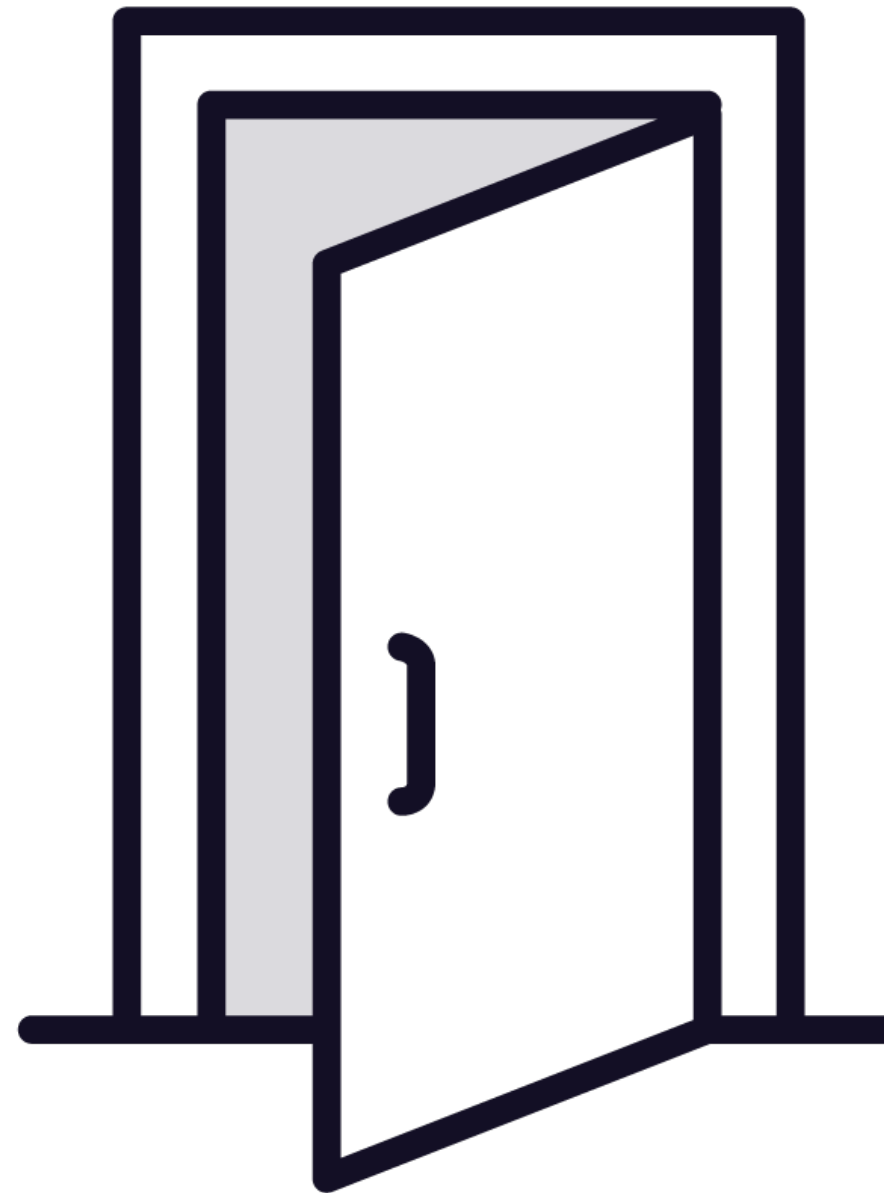
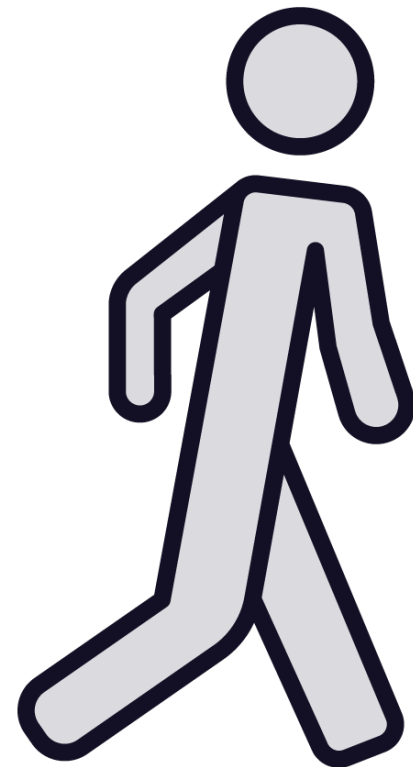
ChildException

```
try {  
    // code  
} catch (ExceptionType1 ex) {  
    // handler  
} catch (ExceptionType2 ex) {  
    // handler  
}
```

Must respect the hierarchy



Understanding Exception Types

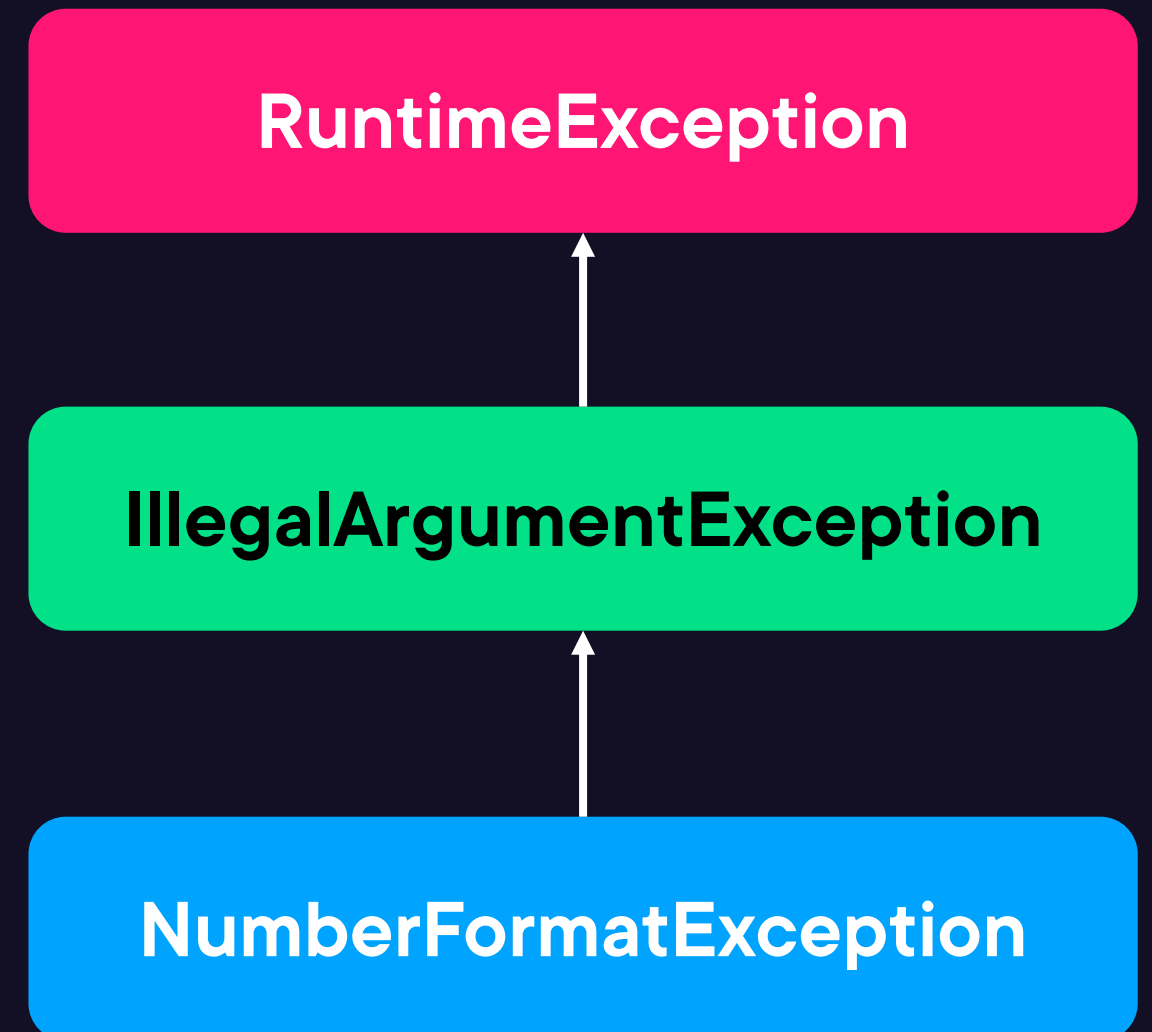


“1a”

“-1”

“Integer.MAX_VALUE + 1”

```
try {  
    // parse user input  
} catch (NumberFormatException ex) {}  
  
catch (IllegalArgumentException ex) {}  
  
catch (RuntimeException ex) {}
```

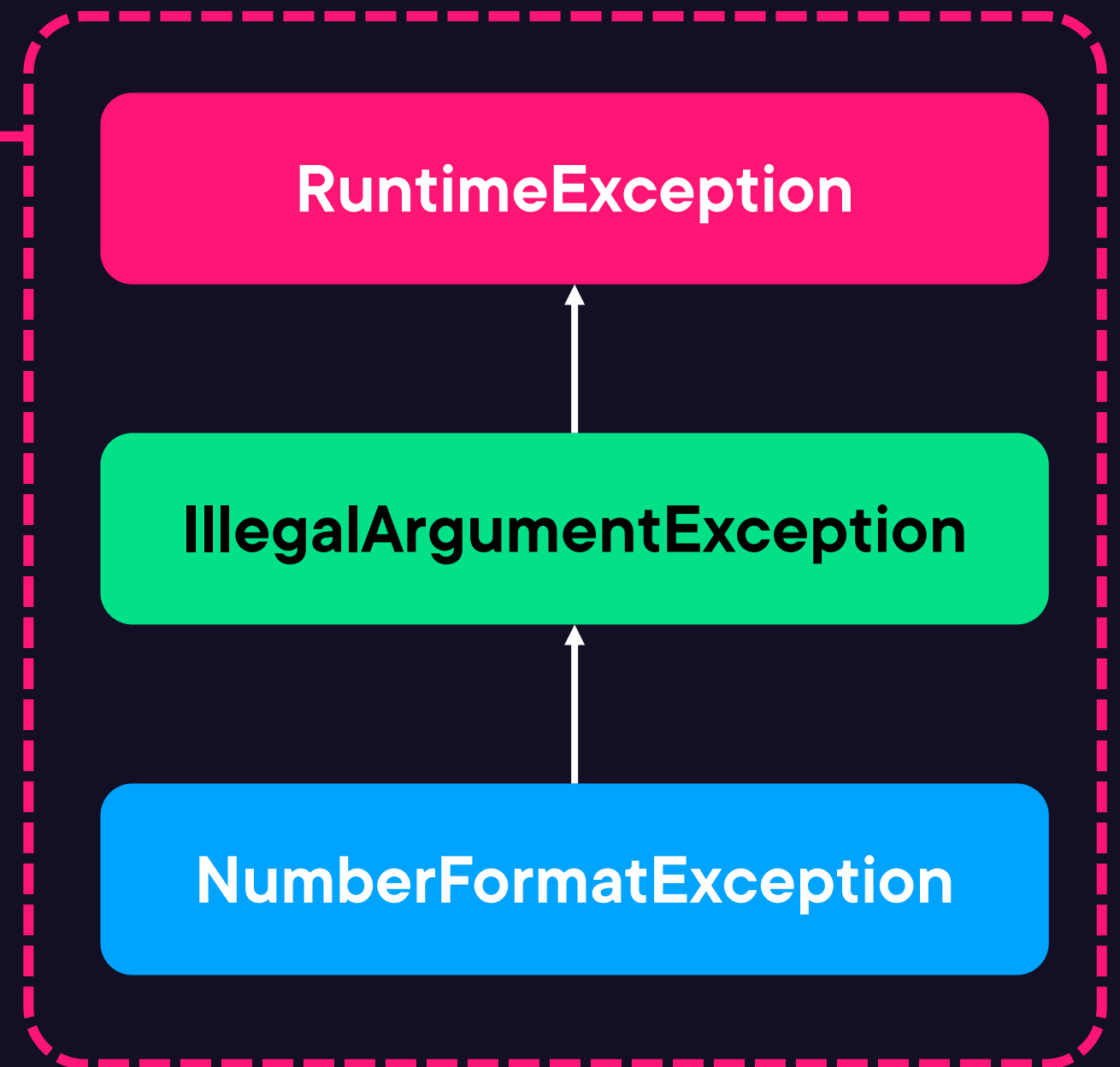


Catches (almost) everything

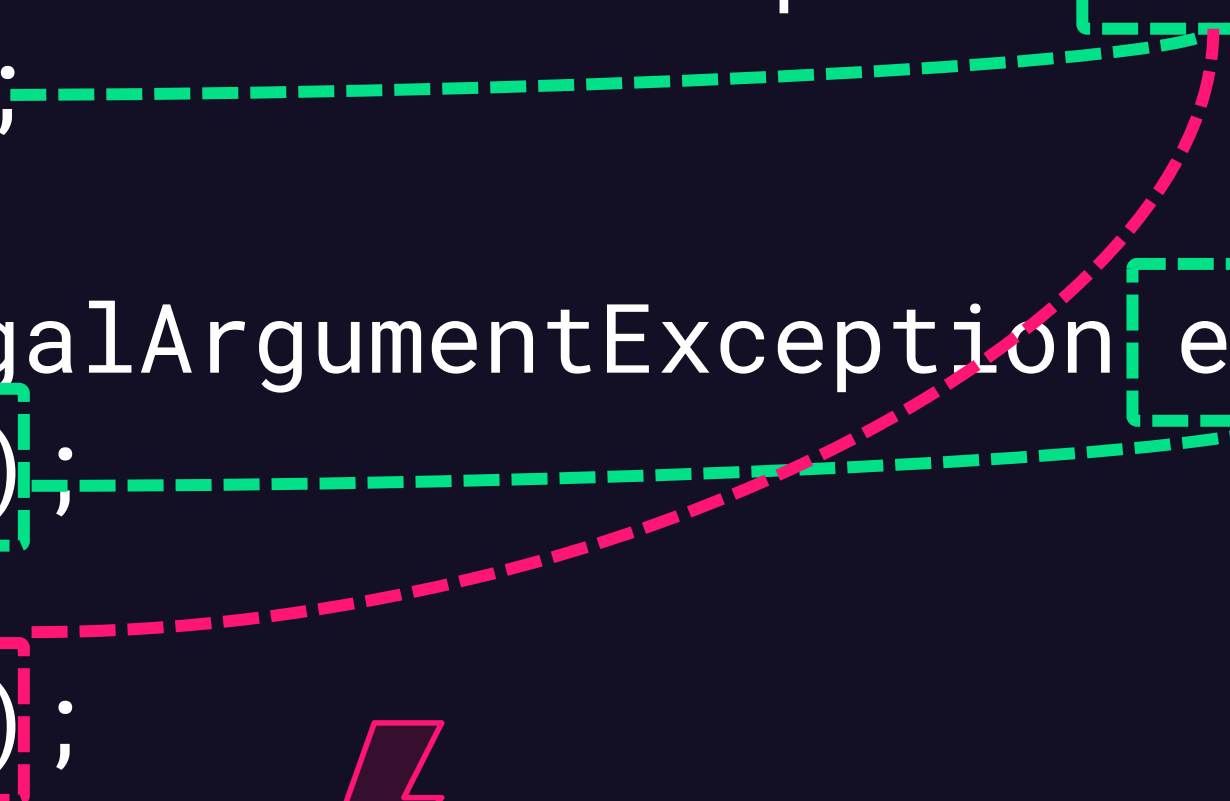
```
try {  
    } catch (RuntimeException ex) {}  
    catch (IllegalArgumentException ex) {}  
    catch (NumberFormatException ex) {}
```



Unreachable statements



```
try {  
    // parse user input  
} catch (NumberFormatException ex1) {  
    log(ex1);  
}  
catch (IllegalArgumentException ex2) {  
    log(ex2);  
  
    log(ex1);  
}
```



```
int num = 0;
try {
    num = Integer.parseInt(args[0]);
}
catch (ArrayIndexOutOfBoundsException ex) { /* same handler */ }

catch (NumberFormatException ex) { /* same handler */ }

catch (IllegalArgumentException ex) { }

catch (RuntimeException ex) { }

System.out.println(num);
```

{ no input }



```
int num = 0;
try {
    num = Integer.parseInt(args[0]);
}

catch (ArrayIndexOutOfBoundsException | NumberFormatException ex)
{
    System.out.println("Missing or invalid input");
}
catch (IllegalArgumentException ex) {
}
catch (RuntimeException ex) { }
System.out.println(num);
```





Compilation failure:
Types in multi-catch must be disjoint

```
int num = 0;
try {
    num = Integer.parseInt(args[0]);
}
catch (ArrayIndexOutOfBoundsException |
      NumberFormatException |
      IllegalArgumentException ex) {
    System.out.println("Missing or invalid input");
}
catch (RuntimeException ex) {
    System.out.println(num);
}
```

IllegalArgumentException

NumberFormatException



```
try {
```

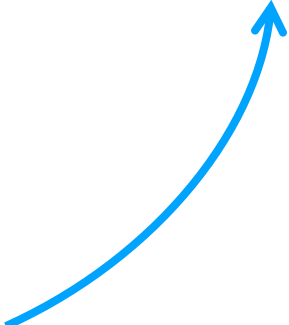
```
    // risky code
```

```
} catch (Exception1 | Exception 2 ex) {
```

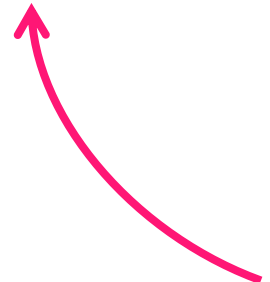
```
    // handler
```

```
}
```

Required between
exception types



Single identifier for
all exceptions



```
catch (Exception1 ex | Exception2 ex)    // no!
```

```
catch (Exception1 ex1 | Exception2 ex2)    // no!
```

```
catch (Exception1 | Exception2 ex)          // yes!
```



Valid Code?

Chained Catches

```
try {  
    } catch (ChildException ex) {  
    } catch (ParentException ex) {  
    }  
}
```



Multi-catch

```
try {  
    } catch (ChildException |  
            ParentException ex) {  
    }  
}
```





Chaining catch() vs. Multi-catch

catch(E1 e) { } catch(E2 e) { }

Handle things differently
Related exceptions allowed
Specific exceptions first

vs.

catch(E1 | E2 e)

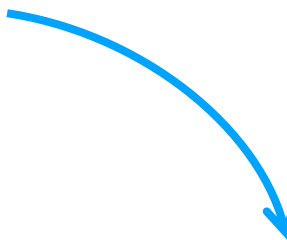
Handle things the same way
Related (non-disjoint) exceptions not allowed



```
try {
```

```
// open file / DB
```

Optional if “finally” is
present



```
} catch (ExceptionType ex) {
```

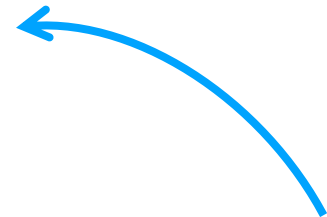
```
// handler
```

```
} finally {
```

```
// close file / DB
```

```
}
```

Always (!) executes



Valid Code?

```
try {  
  } catch (Exception ex) {  
  } finally {  
  }  
}
```

```
try {  
  } finally {  
  }  
}
```



Valid Code?

try?

```
catch (Exception ex) {  
  } finally {  
  }
```

```
try {  
  } finally (Exception ex) {  
  }
```



Stop right now!

```
try {  
    System.exit(0);  
} finally {  
    System.out.println("This won't print");  
}
```

